

Smart Contract Audit

ESIMNFT.sol

Contract:	ESIMNFT
Version:	Solidity ^0.8.28
Chain:	Abstract (testnet 11124 / mainnet 2741)
Date:	2026-07-06
Auditor:	AI-assisted review

This report presents the findings of a security review of the ESIMNFT smart contract. The analysis covers logic correctness, adherence to ERC standards, access control, and potential edge cases in the transferable eSIM voucher system.

Files Reviewed:

contracts/ESIMNFT.sol (268 lines)

Scope:

ERC-721 token with plan metadata (provider, country, data, validity), on-chain activation gating transfers, operator role, EIP-712 permit-based activation, and owner burning of activated tokens.

1 Summary

3 findings were identified: 1 HIGH, 1 LOW, and 1 INFO. The HIGH-severity finding relates to a broken `_exists()` implementation that fails to recognize burned tokens, causing several view functions to return stale data. The remaining findings are minor code-quality issues.

#	Severity	Title	Status
1	HIGH	<code>_exists()</code> ignores burned tokens	Fix proposed
2	LOW	<code>burn()</code> records before state change	Fix proposed
3	LOW	<code>PlanMetadataUpdated</code> event never emitted	Remove
4	INFO	<code>setBaseURI()</code> emits no event	Enhancement
5	INFO	<code>getBurnedTokens()</code> unbounded array	Enhancement

2 Detailed Findings

HIGH

`_exists()` does not account for burned tokens

The `_exists()` function at line 193 uses `'tokenId < _nextTokenId'` to determine existence. After a token is burned, ERC-721's `_burn()` sets `_owners[tokenId]` to `address(0)`, but `_exists()` still returns true. This means `getPlan()`, `isActivated()`, and `tokenURI()` all serve stale data for burned tokens, even though `ownerOf()` correctly reverts. An off-chain caller relying on these functions would receive misleading data.

```
// Current (line 193-195):
function _exists(uint256 tokenId)
    internal view returns (bool) {
    return tokenId < _nextTokenId;
}

// Fix:
function _exists(uint256 tokenId)
    internal view returns (bool) {
    return _ownerOf(tokenId) != address(0);
}
```

LOW

`burn()` records token before `_burn()` succeeds

At line 137, `_burnedTokens.push(tokenId)` executes before `_burn(tokenId)`. If `_burn()` were to revert (e.g., due to reentrancy or a future OZ change), the token would be recorded as burned without being burned. Push after the state-changing call.

```
// Current (line 137-138):
_burnedTokens.push(tokenId);
_burn(tokenId);

// Fix:
_burn(tokenId);
_burnedTokens.push(tokenId);
```

LOW

`PlanMetadataUpdated` event is dead code

The event `PlanMetadataUpdated` is declared at line 54 but no function in the contract ever emits it. This wastes event signature space and misleads readers. Remove the event or add emission in relevant functions.

```
// Line 54 - never emitted:  
event PlanMetadataUpdated(  
    uint256 indexed tokenId,  
    string country,  
    uint256 dataBytes,  
    uint256 validityDays  
);
```

INFO**setBaseURI() emits no event**

The setBaseURI() function at line 175 changes the metadata base URI without emitting any event. Off-chain indexers and dApps that cache tokenURIs have no way to detect this change. Add a BaseURIUpdated event.

```
// Fix:  
event BaseURIUpdated(string uri);  
  
function setBaseURI(string calldata baseURI)  
    external onlyOwner {  
    _baseTokenURI = baseURI;  
    emit BaseURIUpdated(baseURI);  
}
```

INFO**getBurnedTokens() returns unbounded array**

The function at line 166 returns the entire _burnedTokens[] dynamic array. As more tokens are burned, this function grows linearly in gas cost and response size. Consider removing it (the data is available from Transfer events) or adding pagination.

```
// Line 166-168:  
function getBurnedTokens()  
    external view returns (uint256[] memory) {  
    return _burnedTokens;  
}
```

3 Scope & Methodology

The audit covered the full source of contracts/ESIMNFT.sol (268 lines, Solidity 0.8.28). The following areas were analyzed:

- ERC-721 compliance and OpenZeppelin v5 integration
- Access control: onlyOwner, onlyOperator, per-token ownership checks
- Activation lifecycle: mint, transfer, activate, burn sequencing
- EIP-712 typed signature verification in activateWithPermit
- Reentrancy resistance in state-mutating functions
- Token existence and ownership consistency after burns
- Gas optimization and storage layout
- Event emission completeness

The audit is a logic review only. It does not include formal verification, fuzz testing, or economic analysis of the eSIM provider integration. The off-chain components (backend, Vercel API, wallet integration) are out of scope.

4 Recommendations

Fix `_exists()`

HIGH - Use `_ownerOf(tokenId) != address(0)` to correctly reflect token existence after burns. This is the most critical finding.

Reorder `burn()`

LOW - Move `_burn(tokenId)` before `_burnedTokens.push()` to avoid inconsistent state.

Remove dead event

LOW - Delete `PlanMetadataUpdated` unless planned for future use.

Add `BaseURIUpdated` event

INFO - Emit an event in `setBaseURI()` for transparency.

Paginate or remove `getBurnedTokens()`

INFO - The array grows unboundedly with burns.